

qwen3-coder:480b-cloud

FUNCTIONAL REQUIREMENTS DOCUMENT

Freeze-Dried Food — Landing Page

Product Catalog, Detail & Order Booking System

Project	Freeze-Dried Food Landing Page
Document Type	Functional Requirements Specification
Technology Stack	MERN (MongoDB, Express.js, React, Node.js)
Version	1.0 — Initial Draft
Date	May 2026
Status	In Review

1. Introduction

1.1 Purpose

This document defines the functional requirements for a MERN-stack landing page dedicated to selling freeze-dried food products. It covers product listing, individual product detail pages, the order booking flow, administrative management, and user account features.

This specification is intended for use by developers, designers, and project stakeholders to align on scope before development begins.

1.2 Scope

The system will be a web application built using:

- React.js — frontend UI and routing
- Node.js + Express.js — REST API and business logic
- MongoDB + Mongoose — data persistence
- JWT — authentication and session management

The application will allow customers to browse freeze-dried food products, view detailed product information, and place orders online. An admin panel will allow the business owner to manage products and orders.

1.3 Definitions

Term	Definition
FR	Functional Requirement
MERN	MongoDB, Express.js, React, Node.js — the technology stack

Term	Definition
JWT	JSON Web Token — used for stateless authentication
SKU	Stock Keeping Unit — unique identifier per product variant
Admin	Authenticated user with role: 'admin' in the system
Guest	User placing an order without creating an account

2. Product Listing

The product listing section is the primary entry point for customers. It must render all freeze-dried products in a structured, filterable, and searchable catalog.

ID	Requirement	Description	Stack Layer	Priority
FR01	Product catalog display	Render all products in a responsive grid or list view. Each card must show: product image, name, short description, price, and an Add to Cart button.	Frontend	High
FR02	Category filtering	Filter products by category (e.g., Fruits, Vegetables, Full Meals, Snacks, Emergency Kits). Filter state must be reflected in the URL query string for shareability.	Frontend	High
FR03	Price range filter	Allow customers to filter by min/max price using a dual-handle range slider. Updates product results in real time without a page reload.	Frontend	Medium
FR04	Full-text search	Provide a search input that queries product names and descriptions. The API must support a ?search= query parameter for server-side filtering.	Full Stack	High
FR05	Sort options	Sort products by: Price (low to high), Price (high to low), Newest arrivals, Shelf life (longest first), and Popularity (order count).	Frontend	Medium
FR06	Pagination	Implement server-side pagination with configurable page size (default: 12 products/page). Provide page controls and a total count display.	Full Stack	High
FR07	Out-of-stock indicator	Products with stock quantity = 0 must display an 'Out of Stock' badge. The Add to Cart button must be disabled for these products.	Frontend	High

3. Product Detail Page

Each product must have a dedicated detail page accessible via a unique URL (e.g., /products/:id or /products/:slug). This page provides the complete product information needed to support a purchase decision.

ID	Requirement	Description	Stack Layer	Priority
FR08	Product image gallery	Display up to 6 product images in a carousel/lightbox. Thumbnail strip below main image. Images are stored	Frontend	High

Freeze-Dried Food — Functional Requirements				MERN Stack v1.0	
ID	Requirement	Description	Stack Layer	Priority	
FR09	Full product information	externally (Cloudinary recommended) and referenced by URL in MongoDB.			
		Display: full name, complete description, ingredients list, net weight, serving size, number of servings, and preparation instructions.	Full Stack	High	
		Render a structured features block highlighting: shelf life (e.g., 25 years), storage conditions, freeze-drying method, certifications, and country of origin.	Frontend	High	
FR11	Nutritional facts panel	Display a formatted nutritional information table (calories, protein, carbs, fats, sodium) per serving, sourced from the product document in MongoDB.	Full Stack	Medium	
FR12	Variant & quantity selector	Allow selection of pack size variant (e.g., Single, Family Pack, Bulk 12-pack) and numeric quantity (min: 1, max: available stock). Price updates dynamically on variant change.	Frontend	High	
FR13	Add to Cart from detail page	Prominent Add to Cart CTA that adds the selected variant and quantity to the cart. Cart count badge in the navbar must update immediately.	Frontend	High	
FR14	Related products	Display 4 related products based on shared category. Fetched from the backend using a separate API call. Each shows image, name, and price.	Backend	Medium	
FR15	Customer reviews	Show verified purchase reviews with star rating, reviewer name, date, and comment. Display average rating prominently. Logged-in customers can submit reviews.	Full Stack	Medium	

4. Order Booking

The order booking flow converts a customer's cart into a confirmed order stored in MongoDB. The flow must work for both registered users and guest customers.

ID	Requirement	Description	Stack Layer	Priority
FR16	Add to cart	Add products with selected variant and quantity to a persistent cart. Cart state stored in localStorage for guests and synced to the database for logged-in users.	Frontend	High
FR17	Cart management	View cart as a slide-out panel or dedicated page. Support: quantity update (+/-), item removal, clear cart. Display line item totals and running subtotal.	Frontend	High
FR18	Checkout form	Collect: full name, email address, phone number, shipping address (line 1, line 2, city, state, PIN code), and preferred delivery date.	Frontend	High
FR19	Client-side validation	Validate all checkout form fields in real time: required fields, email format, 10-digit phone, valid PIN code. Display inline error messages per field.	Frontend	High
FR20	Server-side validation	Re-validate all order data on the backend before persisting. Return structured error responses (HTTP 422) for any invalid fields.	Backend	High
FR21	Order persistence	Save confirmed orders to MongoDB with: order ID, customer info, line items (product ID, variant, qty, unit price), totals, status, and timestamps.	Database	High
FR22	Order confirmation page	Display a success page with the order ID, itemised summary, and estimated delivery date. Provide a 'Continue Shopping' CTA.	Frontend	High
FR23	Confirmation email	Send a transactional email to the customer with the order summary, order ID, and contact information. Use Nodemailer with an SMTP provider (e.g., SendGrid).	Backend	High
FR24	Order status lookup	Allow any customer to enter their Order ID and email on a public page to retrieve current order status without logging in.	Full Stack	Medium
FR25	Stock decrement on order	Reduce the product variant's stock quantity in MongoDB when an order is	Backend	High

Freeze-Dried Food — Functional Requirements				MERN Stack v1.0
ID	Requirement	Description	Stack Layer	Priority
		confirmed. Prevent order submission if stock is insufficient at time of checkout.		

5. Admin Panel

A protected section accessible only to users with the 'admin' role. The admin panel enables full product management, order management, and inventory control.

ID	Requirement	Description	Stack Layer	Priority
FR26	Admin authentication	Admin routes must be protected by JWT middleware that validates the user's role. Non-admin users receive HTTP 403. A dedicated /admin login page is required.	Auth	High
FR27	Create product	Form to add a new product with all fields: name, description, categories, variants (size, price, stock), images (upload to Cloudinary), features, and nutritional data.	Full Stack	High
FR28	Edit product	Pre-populated form to edit any existing product. Changes must persist to MongoDB. Image gallery must support adding and removing images individually.	Full Stack	High
FR29	Delete product	Soft-delete products (set isDeleted: true) so existing orders referencing the product remain intact. Deleted products are hidden from the public catalog.	Backend	Medium
FR30	Order management dashboard	Table listing all orders with: Order ID, customer name, order date, total, and status. Support filtering by status and sorting by date.	Full Stack	High
FR31	Update order status	Admin can update order status through: Placed → Processing → Shipped → Delivered. Status change triggers an email notification to the customer.	Backend	High
FR32	Inventory management	View all products with current stock levels. Edit stock quantity directly from the inventory table. Items below a configurable low-stock threshold are highlighted.	Full Stack	Medium
FR33	Export orders to CSV	Admin can export the current filtered order list as a CSV file for use in	Backend	Low

ID	Requirement	Description	Stack Layer	Priority
		external tools. The backend generates the file on demand.		

6. Authentication & User Accounts

The system supports both registered user accounts and guest checkout. Authentication uses JWT tokens stored in HTTP-only cookies for security.

ID	Requirement	Description	Stack Layer	Priority
FR34	User registration	Registration form with: name, email, password (min 8 chars, with strength indicator), and password confirmation. Passwords stored as bcrypt hashes in MongoDB.	Auth	High
FR35	User login	Email + password login. On success, issue a JWT stored in an HTTP-only cookie. Provide 'Forgot password' link and a 'Remember me' option (30-day session).	Auth	High
FR36	Google OAuth (optional)	Allow login/signup via Google OAuth 2.0 using Passport.js. Creates a user record on first login. Merges with existing email account if the email matches.	Auth	Medium
FR37	Guest checkout	Customers can complete checkout without an account. After order confirmation, prompt the guest to create an account to track their order more easily.	Frontend	High
FR38	Order history	Logged-in users can view all past orders in their account dashboard. Each order shows status, date, items, and total. A 'Reorder' button re-adds items to cart.	Full Stack	Medium
FR39	Profile management	Users can update their name, email, phone, and saved delivery addresses. Password change requires confirmation of the current password.	Full Stack	Low
FR40	Password reset	Forgot password flow: enter email → receive a time-limited reset link (1 hour) → set new password. Reset tokens stored as hashed values in MongoDB.	Backend	Medium

7. Landing Page & UX

The landing page is the first impression for new visitors. It must communicate brand value, surface featured products, and guide visitors toward making a purchase.

Freeze-Dried Food — Functional Requirements				MERN Stack v1.0
ID	Requirement	Description	Stack Layer	Priority
FR41	Hero section	Full-width banner with brand headline, sub-headline, a featured product image, and a primary CTA button ('Shop Now'). Content must be editable by admin.	Frontend	High
FR42	Key benefits section	Icon-and-copy grid highlighting: 25-year shelf life, lightweight and portable, no preservatives or additives, nutrition retention, and easy rehydration.	Frontend	High
FR43	Featured products section	Horizontally scrollable row of admin-selected featured products on the homepage. Configurable from the admin panel (mark product as 'featured').	Full Stack	Medium
FR44	Customer testimonials	Display 3–6 customer reviews on the homepage, pulled from the reviews collection. Admin can mark specific reviews as 'featured' for homepage display.	Full Stack	Medium
FR45	Enquiry / contact form	Form for bulk order enquiries: name, email, phone, message, and estimated quantity. Submission sends an email to the admin via backend mailer.	Backend	Medium
FR46	Responsive design	Full responsiveness across mobile (360px+), tablet (768px+), and desktop (1280px+). Navigation collapses to a hamburger menu on mobile.	Frontend	High
FR47	SEO fundamentals	Each page must render unique <title> and <meta description> tags. Product pages include structured data (JSON-LD) for product schema. React Helmet or equivalent.	Frontend	Medium

8. Non-Functional Considerations

The following non-functional aspects should be planned for during development. These are not functional requirements but are critical for production readiness.

Area	Recommendation
Image Storage	Use Cloudinary for product image uploads. Store only the public_id and URL in MongoDB. Avoid storing binaries in the database.
Payment Gateway	Integrate Razorpay (recommended for India) or Stripe. Never handle raw card data on your server — use the gateway's hosted fields.
API Security	Apply rate limiting (express-rate-limit), CORS whitelist, Helmet.js headers, and input sanitisation (express-validator) on all public endpoints.
Performance	Index MongoDB on: category, price, isDeleted, and createdAt fields. Use React.lazy for code splitting. Serve images at multiple sizes via Cloudinary transforms.
Environment Config	All secrets (JWT secret, DB URI, Cloudinary keys, SMTP credentials) must be stored in environment variables. Never commit .env to version control.
Deployment	Frontend: Vercel or Netlify. Backend: Railway, Render, or a VPS. Database: MongoDB Atlas. Use separate environments for development and production.

9. Requirement Summary

The following table summarises requirement counts by section and priority.

Section	Total	High	Medium	Low
Product Listing	7	4	3	0
Product Detail Page	8	5	3	0
Order Booking	10	8	2	0
Admin Panel	8	4	3	1
Authentication & Accounts	7	3	3	1
Landing Page & UX	7	3	4	0
TOTAL	47	27	18	2

— End of Document —